A rotary self-balancing pendulum visualization through the implementation of LQR control

Philip Tjuatja

October 12, 2026

Contents

1 Introduction and motivation **3**

2 Dynamics of the inverted pendulum **3**
2.1 Derivation of EoM 3
2.1.1 Define origin and bases 4
2.1.2 Kinematics 5
2.1.3 Forces, Moments and Energy 6
2.1.4 Equations of Motion 6
2.2 State-space model 8

3 Control theory **10**
3.1 Consideration on the use of LQR instead of PID control 10
3.2 Derivation of LQR theory 10
3.2.1 Infinite horizon continuous-time case 10
3.2.2 Infinite horizon discrete-time case 11
3.3 Solving for the optimal control law 13
3.3.1 The sampled plant 13
3.3.2 The Riccati map and Riccati iteration 13
3.3.3 Convergence and stability of the iteration 14
3.3.4 Termination 14

4 Software implementation **16**
4.1 Software layout 16
4.2 Linearisation 16
4.3 Zero-order-hold discretisation 17
4.4 The Riccati iteration in code 17
4.5 Design pipeline and the two designs 18
4.5.1 The control rate and the design 19
4.6 Control loop implementation and energy-pumping swing up 19
4.7 Verification of the default gains 20
4.7.1 Synthesiser properties 20
4.7.2 Termination and agreement with a direct solve 21
4.7.3 Plant and controller tests 21

Acknowledgements **23**

References **24**

1 Introduction and motivation

This project aims to visualize the behavior of a rotary self-balancing pendulum through the implementation of Linear Quadratic Regulator (LQR) control. The system under consideration is an inverted rotary pendulum, a classic benchmark in nonlinear control theory, first introduced by Furuta, Yamakita and Kobayashi [1]. The non-linear dynamics of this system make it a challenging yet interesting case study for control engineering, which is a field of interest of mine. I have chosen to implement LQR control for this system due to its optimality properties and its ability to handle multi-variable systems effectively.

I developed an interest in the topics surrounding this project as after first taking my Intro. to Dynamics and Vibration course (ME2400J) in my university. As a fresh student, visualizing where the math goes and how it translates or connects to real life applications could be tricky at first. Because of this, this project aims to give a clear and intuitive understanding of the system's behavior through visualization of a classical control system. This document will outline the derivation of the equations of motion for the pendulum system, linearization of them around the upright equilibrium point, design of an LQR controller to balance the system, and visualization of the system's response under this control strategy. This is done in hopes that it will give a "goal" for future students stepping into this subject matter, and hence, motivated to learn more about dynamics and its applications in fields like control systems.

2 Dynamics of the inverted pendulum

This section aims to derive the model for the pendulum system. I have formulated the question in §2.1 that is to be solved in a similar fashion to one that I am familiar with when undergoing my university's Intro. to Dynamics and Vibration course (ME2400J), taught by Prof. Robert G. Parker. The following derivation is done through the methods introduced in the instructor's Essential Dynamics handout [2], specifically from the section for the treatment of Lagrange's equations in Chapter 5: Analytical Dynamics.

2.1 Derivation of EoM

This is a balancing inverted rotary pendulum where a variable moment $M(t)$ is applied about the origin O as shown to keep the pendulum upright. Treatment of these bars are to be rigid bodies; derivations are to neglect dry friction on bearings and viscous drag. Lengths of respective bars are l_A, l_B. Gravity acts in the $-\underset{\sim}{\mathbf{E}}_3$ direction. The following are the inertia matrices for both bars about their respective points of reference.

$$
\begin{aligned}
{}^{B}\underline{\mathbf{J}}^{O}{}_{\{\underset{\sim}{\mathbf{e}}'_i\}} &= \operatorname{diag}({}^{B}J_1, {}^{B}J_2, {}^{B}J_3) \\
{}^{A}\underline{\mathbf{J}}^{S}{}_{\{\underset{\sim}{\mathbf{e}}_i\}} &= \operatorname{diag}({}^{A}J_1, {}^{A}J_2, {}^{A}J_3)
\end{aligned}
$$

Find EoMs for the inverted rotary pendulum.

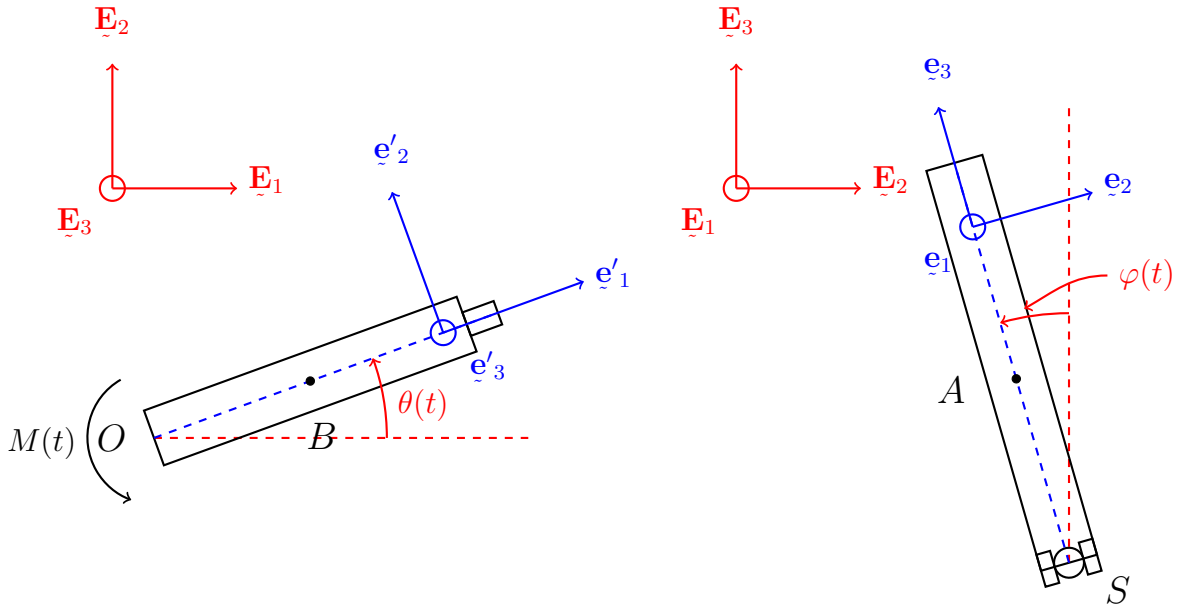


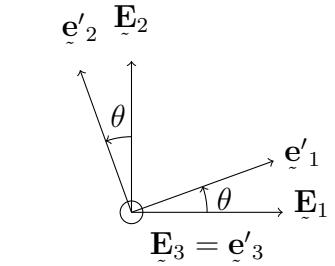
Figure 1: Sketches for the (*left*) top view and (*right*) front view of the rotary inverted pendulum, with bases defined attached.

2.1.1 Define origin and bases

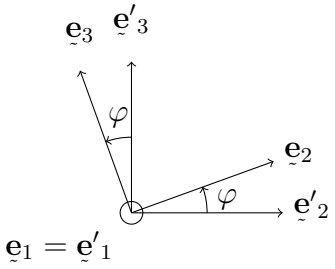
We first define the origin O as shown on the figure. The following bases are defined to denote the rotations due to the joints in the arms:

- $\{\underset{\sim}{\mathbf{E}}_i\}$: Fixed basis as shown.
- $\{\underset{\sim}{\mathbf{e}}'_i\}$: Rotate $\{\underset{\sim}{\mathbf{E}}_i\}$ by angle θ about $\underset{\sim}{\mathbf{E}}_3$
- $\{\underset{\sim}{\mathbf{e}}_i\}$: Rotate $\{\underset{\sim}{\mathbf{e}}'_i\}$ by angle φ about $\underset{\sim}{\mathbf{e}}'_1$

Following this, we move away from the usual derivation of the angular velocity matrix and time derivative of bases by means of the rotation matrices and instead utilize the ideas highlighted in Chapter 4.6 of the Essential Dynamics handout [2]. The angular velocity of the system is given by the addition of the angular velocities of the two rotations, which are expressed in their respective bases. The angular velocity of the system is then expressed in the $\{\underset{\sim}{\mathbf{e}}_i\}$ basis as follows, alongside the time derivatives of the basis vectors $\underset{\sim}{\mathbf{e}}_i$:



$$\underset{\sim}{\boldsymbol{\omega}} = \dot{\theta}\underset{\sim}{\mathbf{e}}'_3 + \dot{\varphi}\underset{\sim}{\mathbf{e}}_1$$
$$= \dot{\varphi}\underset{\sim}{\mathbf{e}}_1 + \dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_2 + \dot{\theta}\cos\varphi\underset{\sim}{\mathbf{e}}_3$$



$$\dot{\underset{\sim}{\mathbf{e}}}_1 = \underset{\sim}{\boldsymbol{\omega}} \times \underset{\sim}{\mathbf{e}}_1 = -\dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_3 + \dot{\theta}\cos\varphi\underset{\sim}{\mathbf{e}}_2$$
$$\dot{\underset{\sim}{\mathbf{e}}}_2 = \underset{\sim}{\boldsymbol{\omega}} \times \underset{\sim}{\mathbf{e}}_2 = \dot{\varphi}\underset{\sim}{\mathbf{e}}_3 - \dot{\theta}\cos\varphi\underset{\sim}{\mathbf{e}}_1$$
$$\dot{\underset{\sim}{\mathbf{e}}}_3 = \underset{\sim}{\boldsymbol{\omega}} \times \underset{\sim}{\mathbf{e}}_3 = -\dot{\varphi}\underset{\sim}{\mathbf{e}}_2 + \dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_1$$

2.1.2 Kinematics

Here, we calculate the position, velocity and angular velocities of the centers of mass of both bars. The dot product quantities are calculated to be used in the kinetic energy expression in the next subsection.

So firstly, for the center of mass of bar B:

$$\underset{\sim}{\mathbf{r}}_B = \frac{l_B}{2}\underset{\sim}{\mathbf{e}}'_1$$
$$= \frac{l_B}{2}\underset{\sim}{\mathbf{e}}_1$$
$$\dot{\underset{\sim}{\mathbf{r}}}_B = \frac{l_B}{2}\dot{\underset{\sim}{\mathbf{e}}}_1$$
$$= \frac{l_B}{2}(-\dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_3 + \dot{\theta}\cos\varphi\underset{\sim}{\mathbf{e}}_2)$$
$$\dot{\underset{\sim}{\mathbf{r}}}_B \cdot \dot{\underset{\sim}{\mathbf{r}}}_B = \frac{l_B^2}{4}\dot{\theta}^2$$

$$^B\underset{\sim}{\boldsymbol{\omega}} = \dot{\theta}\underset{\sim}{\mathbf{e}}'_3$$

$$^B\underset{\sim}{\boldsymbol{\omega}} \cdot {}^B\underline{\mathbf{J}} \cdot {}^B\underset{\sim}{\boldsymbol{\omega}} = {}^BJ_3\dot{\theta}^2$$

and using the previous values, for the center of mass of bar A:

$$\begin{aligned}
\underset{\sim}{\mathbf{r}}_A &= \underset{\sim}{\mathbf{r}}_B + \frac{l_A}{2}\underset{\sim}{\mathbf{e}}_3 \\
&= l_B\underset{\sim}{\mathbf{e}}_1 + \frac{l_A}{2}\underset{\sim}{\mathbf{e}}_3 \\
\dot{\underset{\sim}{\mathbf{r}}}_A &= l_B(-\dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_3 + \dot{\theta}\cos\varphi\underset{\sim}{\mathbf{e}}_2) + \frac{l_A}{2}(-\dot{\varphi}\underset{\sim}{\mathbf{e}}_2 + \dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_1) \\
&= \frac{l_A}{2}\dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_1 + \left[-\frac{l_A}{2}\dot{\varphi} + l_B\dot{\theta}\cos\varphi\right]\underset{\sim}{\mathbf{e}}_2 - l_B\dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_3 \\
\dot{\underset{\sim}{\mathbf{r}}}_A \cdot \dot{\underset{\sim}{\mathbf{r}}}_A &= \frac{l_A^2}{4}\dot{\theta}^2\sin^2\varphi + \frac{l_A^2}{4}\dot{\varphi}^2 + l_B^2\dot{\theta}^2\cos^2\varphi + l_B^2\dot{\theta}^2\sin^2\varphi - l_A l_B\dot{\theta}\dot{\varphi}\cos\varphi \\
&= \frac{l_A^2}{4}[\dot{\theta}^2\sin^2\varphi + \dot{\varphi}^2] + l_B^2\dot{\theta}^2 - l_A l_B\dot{\theta}\dot{\varphi}\cos\varphi \\
{}^A\underset{\sim}{\boldsymbol{\omega}} &= \dot{\varphi}\underset{\sim}{\mathbf{e}}_1 + \dot{\theta}\sin\varphi\underset{\sim}{\mathbf{e}}_2 + \dot{\theta}\cos\varphi\underset{\sim}{\mathbf{e}}_3 \\
{}^A\underset{\sim}{\boldsymbol{\omega}} \cdot {}^A\underline{\mathbf{J}} \cdot {}^A\underset{\sim}{\boldsymbol{\omega}} &= {}^AJ_1\dot{\varphi}^2 + {}^AJ_2\dot{\theta}^2\sin^2\varphi + {}^AJ_3\dot{\theta}^2\cos^2\varphi
\end{aligned}$$

2.1.3 Forces, Moments and Energy

Since I chose to use Lagrange's equations to obtain the EoMs, the forces acting on the system are categorized into three groups: conservative, constraint and non-conservative forces/moments. Conservative forces such that there exist a respective potential:

- Gravity: $-m_A g\underset{\sim}{\mathbf{E}}_3$ on Bar A and $-m_B g\underset{\sim}{\mathbf{E}}_3$ on Bar B. It should be known that there is no contribution from B since there is no change in $\underset{\sim}{\mathbf{r}}_B$ in the $\underset{\sim}{\mathbf{E}}_3$ direction.

There are constraint forces acting on both bars, where identifying them would have been useful for a derivation through balances of linear and angular momentum. However, as these constraints are time-independent, they do not do virtual (and real) work. In this case, we assume for the EoM that there are no non-conservative forces acting on the system. However, there is a non-conservative moment acted on origin O, that is, the supplied moment by actuator (due to control system for balancing): $\underset{\sim}{\mathbf{M}} = M(t)\underset{\sim}{\mathbf{E}}_3$.

So the kinetic and potential energies of the system are given by:

$$\begin{aligned}
T &= \frac{1}{2}m_A\dot{\underset{\sim}{\mathbf{r}}}_A \cdot \dot{\underset{\sim}{\mathbf{r}}}_A + \frac{1}{2}m_B\dot{\underset{\sim}{\mathbf{r}}}_B \cdot \dot{\underset{\sim}{\mathbf{r}}}_B + \frac{1}{2}{}^A\underset{\sim}{\boldsymbol{\omega}} \cdot {}^A\underline{\mathbf{J}} \cdot {}^A\underset{\sim}{\boldsymbol{\omega}} + \frac{1}{2}{}^B\underset{\sim}{\boldsymbol{\omega}} \cdot {}^B\underline{\mathbf{J}} \cdot {}^B\underset{\sim}{\boldsymbol{\omega}} \\
&= \frac{1}{2}m_A\left(\frac{l_A^2}{4}[\dot{\theta}^2\sin^2\varphi + \dot{\varphi}^2] + l_B^2\dot{\theta}^2 - l_A l_B\dot{\theta}\dot{\varphi}\cos\varphi\right) + \frac{1}{2}m_B\frac{l_B^2}{4}\dot{\theta}^2 \\
&\quad + \frac{1}{2}\left({}^AJ_1\dot{\varphi}^2 + \left[{}^AJ_2\sin^2\varphi + {}^AJ_3\cos^2\varphi + {}^BJ_3\right]\dot{\theta}^2\right) \\
V &= m_A g\underset{\sim}{\mathbf{E}}_3 \cdot \underset{\sim}{\mathbf{r}}_A = m_A g\,\underset{\sim}{\mathbf{r}}_A \cdot (\sin\varphi\underset{\sim}{\mathbf{e}}_2 + \cos\varphi\underset{\sim}{\mathbf{e}}_3) \\
&= m_A g\frac{l_A}{2}\cos\varphi
\end{aligned}$$

2.1.4 Equations of Motion

Using classical Lagrange's equations:

$$\frac{d}{dt}\frac{\partial T}{\partial\dot{q}} - \frac{\partial T}{\partial q} + \frac{\partial V}{\partial q} = \sum\left(\underset{\sim}{\mathbf{F}}^{nc} \cdot \frac{\partial\underset{\sim}{\mathbf{r}}}{\partial q} + \underline{\mathbf{M}}^{nc} \cdot \frac{\partial\underset{\sim}{\boldsymbol{\omega}}}{\partial\dot{q}}\right)$$

Applying to θ:

$$\begin{aligned}
\frac{\partial T}{\partial \dot{\theta}} &= \frac{1}{2} m_A \left(\frac{l_A^2}{4} 2\dot{\theta} \sin^2\varphi + l_B^2 2\dot{\theta} - l_A l_B \dot{\varphi} \cos\varphi \right) + \frac{1}{2} m_B \frac{l_B^2}{4} 2\dot{\theta} \\
&\quad + \left({}^A J_2 \sin^2\varphi + {}^A J_3 \cos^2\varphi + {}^B J_3 \right) \dot{\theta} \\
\frac{d}{dt}\frac{\partial T}{\partial \dot{\theta}} &= \frac{l_A^2}{4} m_A \ddot{\theta} \sin^2\varphi + \frac{l_A^2}{4} m_A \dot{\theta}\dot{\varphi} 2 \sin\varphi\cos\varphi + l_B^2 m_A \ddot{\theta} + m_B \frac{l_B^2}{4} \ddot{\theta} \\
&\quad - \frac{l_A l_B}{2} m_A [\ddot{\varphi}\cos\varphi - \dot{\varphi}^2 \sin\varphi] + \left({}^A J_2 \sin^2\varphi + {}^A J_3 \cos^2\varphi + {}^B J_3 \right) \ddot{\theta} \\
&= \left[m_A \left(\frac{l_A^2}{4} \sin^2\varphi + l_B^2 \right) + m_B \frac{l_B^2}{4} \right] \ddot{\theta} + \left[\frac{l_A^2}{2} m_A \sin\varphi\cos\varphi \right] \dot{\theta}\dot{\varphi} \\
&\quad + \left[{}^A J_2 \sin^2\varphi + {}^A J_3 \cos^2\varphi + {}^B J_3 \right] \ddot{\theta} \\
&\quad - m_A \left[\left(\frac{l_A l_B}{2} \cos\varphi \right) \ddot{\varphi} - \left(\frac{l_A l_B}{2} \sin\varphi \right) \dot{\varphi}^2 \right] \\
\frac{\partial T}{\partial \theta} &= 0 \quad ; \quad \frac{\partial V}{\partial \theta} = 0 \quad ; \quad (M \sin\varphi \underset{\sim}{\mathbf{e}}_2 + M\cos\varphi \underset{\sim}{\mathbf{e}}_3) \cdot (\sin\varphi \underset{\sim}{\mathbf{e}}_2 + \cos\varphi \underset{\sim}{\mathbf{e}}_3) = M
\end{aligned}$$

$$\begin{aligned}
\Rightarrow & \left[m_A \left(\frac{l_A^2}{4} \sin^2\varphi + l_B^2 \right) + m_B \frac{l_B^2}{4} \right] \ddot{\theta} + \left[\frac{l_A^2}{2} m_A \sin\varphi\cos\varphi \right] \dot{\theta}\dot{\varphi} \\
& + \left[{}^A J_2 \sin^2\varphi + {}^A J_3 \cos^2\varphi + {}^B J_3 \right] \ddot{\theta} \\
& - m_A \left[\left(\frac{l_A l_B}{2} \cos\varphi \right) \ddot{\varphi} - \left(\frac{l_A l_B}{2} \sin\varphi \right) \dot{\varphi}^2 \right] = M(t)
\end{aligned}$$

and also for φ:

$$\begin{aligned}
\frac{\partial T}{\partial \dot{\varphi}} &= m_A \frac{l_A^2}{4} \dot{\varphi} + {}^A J_1 \dot{\varphi} - m_A \frac{l_A l_B}{2} \dot{\theta} \cos\varphi \\
\frac{d}{dt}\frac{\partial T}{\partial \dot{\varphi}} &= \left[m_A \frac{l_A^2}{4} + {}^A J_1 \right] \ddot{\varphi} - \left[\left(m_A \frac{l_A l_B}{2} \ddot{\theta} \cos\varphi \right) - \left(m_A \frac{l_A l_B}{2} \dot{\theta}\dot{\varphi} \sin\varphi \right) \right] \\
\frac{\partial T}{\partial \varphi} &= \frac{1}{2} m_A \frac{l_A^2}{4} \dot{\theta}^2 \cdot 2 \sin\varphi\cos\varphi + m_A \frac{l_A l_B}{2} \dot{\theta}\dot{\varphi} \sin\varphi \\
&\quad + {}^A J_2 \dot{\theta}^2 \sin\varphi\cos\varphi - {}^A J_3 \dot{\theta}^2 \cos\varphi\sin\varphi \\
\frac{\partial V}{\partial \varphi} &= -m_A g \frac{l_A}{2} \sin\varphi \quad ; \quad (M \sin\varphi \underset{\sim}{\mathbf{e}}_2 + M\cos\varphi \underset{\sim}{\mathbf{e}}_3) \cdot (\underset{\sim}{\mathbf{e}}_1) = 0
\end{aligned}$$

$$\begin{aligned}
\Rightarrow & \left[m_A \frac{l_A^2}{4} + {}^A J_1 \right] \ddot{\varphi} - \left[m_A \frac{l_A^2}{4} + {}^A J_2 - {}^A J_3 \right] \dot{\theta}^2 \sin\varphi\cos\varphi \\
& - \left(m_A \frac{l_A l_B}{2} \right) \ddot{\theta} \cos\varphi - m_A g \frac{l_A}{2} \sin\varphi = 0
\end{aligned}$$

Hence, the following equations are the model for the rotary inverted pendulum sys-

tem:

$$\begin{aligned}
&\left[m_A\left(\frac{l_A^2}{4}\sin^2\varphi + l_B^2\right) + m_B\frac{l_B^2}{4}\right]\ddot{\theta} + \left[\frac{l_A^2}{2}m_A\sin\varphi\cos\varphi\right]\dot{\theta}\dot{\varphi} \\
&\quad + \left[{}^A J_2\sin^2\varphi + {}^A J_3\cos^2\varphi + {}^B J_3\right]\ddot{\theta} \\
&\quad - m_A\left[\left(\frac{l_A l_B}{2}\cos\varphi\right)\ddot{\varphi} - \left(\frac{l_A l_B}{2}\sin\varphi\right)\dot{\varphi}^2\right] \qquad = M(t) \\
&\left[m_A\frac{l_A^2}{4} + {}^A J_1\right]\ddot{\varphi} - \left[m_A\frac{l_A^2}{4} + {}^A J_2 - {}^A J_3\right]\dot{\theta}^2\sin\varphi\cos\varphi \\
&\quad - \left(m_A\frac{l_A l_B}{2}\right)\ddot{\theta}\cos\varphi \qquad = m_A g\frac{l_A}{2}\sin\varphi
\end{aligned} \tag{1}$$

The model is complete since there are 2 unknowns θ, φ and 2 equations in the model.

2.2 State-space model

Rewriting (1) in matrix form yields the equation

$$\underline{\mathbf{M}}\ddot{\underset{\sim}{\mathbf{q}}} + \underline{\mathbf{C}}\dot{\underset{\sim}{\mathbf{q}}} = \underset{\sim}{\boldsymbol{\tau}}$$

where, by denoting the following terms for simplicity:

$$\begin{aligned}
J_\theta &= m_A l_B^2 + m_B\frac{l_B^2}{4} + {}^B J_3 + {}^A J_3, & J_\varphi &= m_A\frac{l_A^2}{4} + {}^A J_1, \\
B &= m_A\frac{l_A^2}{4} + {}^A J_2 - {}^A J_3, & L &= \frac{l_A l_B}{2}m_A,
\end{aligned}$$

we obtain the mass matrix $\underline{\mathbf{M}}(\underset{\sim}{\mathbf{q}})$, Christoffel matrix $\underline{\mathbf{C}}(\underset{\sim}{\mathbf{q}}, \dot{\underset{\sim}{\mathbf{q}}})$ and moment vector $\underset{\sim}{\boldsymbol{\tau}}(\underset{\sim}{\mathbf{q}}, t)$ of the system, given in terms of the modal coordinate $\underset{\sim}{\mathbf{q}}$:

$$\underline{\mathbf{M}}, \underline{\mathbf{C}} \in \mathcal{M}_2(\mathbb{R}), \quad \underset{\sim}{\mathbf{q}} = \begin{bmatrix}\varphi \\ \theta\end{bmatrix}; \quad \underset{\sim}{\boldsymbol{\tau}} = \begin{bmatrix}M(t) \\ m_A g\frac{l_A}{2}\sin\varphi\end{bmatrix}$$

$$\underline{\mathbf{M}} = \begin{bmatrix}J_\theta + B\sin^2\varphi & -L\cos\varphi \\ -L\cos\varphi & J_\varphi\end{bmatrix}$$

$$\underline{\mathbf{C}} = \begin{bmatrix}B\dot{\varphi}\sin\varphi\cos\varphi & B\dot{\theta}\sin\varphi\cos\varphi + L\dot{\varphi}\sin\varphi \\ -B\dot{\theta}\sin\varphi\cos\varphi & 0\end{bmatrix}$$

Linearizing at the upright equilibrium, that is $\underset{\sim}{\mathbf{q}}_0 = \dot{\underset{\sim}{\mathbf{q}}}_0 = \underset{\sim}{\mathbf{0}}$, the matrices yield

$$\underline{\mathbf{M}}_0 \equiv \underline{\mathbf{M}}(0) = \begin{bmatrix}J_\theta & -L \\ -L & J_\varphi\end{bmatrix}, \qquad \underline{\mathbf{C}}(0,0) = \underline{\mathbf{0}},$$

and since $\sin\varphi \approx \varphi$ for small φ, the linearized torque vector is

$$\underset{\sim}{\boldsymbol{\tau}} \approx \begin{bmatrix}M(t) \\ m_A g\frac{l_A}{2}\end{bmatrix}.$$

Since $\underline{\mathbf{C}}$ is quadratic in its terms, it vanishes in the first-order approximation. So, the equilibrium state yields

$$\begin{bmatrix} J_\theta & -L \\ -L & J_\varphi \end{bmatrix} \begin{bmatrix} \ddot{\theta} \\ \ddot{\varphi} \end{bmatrix} = \begin{bmatrix} M(t) \\ \dfrac{m_A g l_A}{2}\,\varphi \end{bmatrix}$$

For $\det \underline{\mathbf{M}}_0 = \Delta = J_\theta J_\varphi - L^2 \neq 0$, $\underline{\mathbf{M}}_0$ is invertible and thus obtaining the equation

$$\begin{bmatrix} \ddot{\theta} \\ \ddot{\varphi} \end{bmatrix} = \frac{1}{\Delta} \begin{bmatrix} J_\varphi & L \\ L & J_\theta \end{bmatrix} \begin{bmatrix} M(t) \\ \dfrac{m_A g l_A}{2}\,\varphi \end{bmatrix}.$$

which after multiplying, yields the scalar equations

$$\ddot{\varphi} = \frac{L}{\Delta} M(t) + \frac{J_\theta m_A g l_A}{2\Delta}\,\varphi$$
$$\ddot{\theta} = \frac{J_\varphi}{\Delta} M(t) + \frac{m_A g l_A L}{2\Delta}\,\varphi$$

Using these equations and defining the state vector $\underset{\sim}{\mathbf{x}} = [\varphi \quad \theta \quad \dot{\varphi} \quad \dot{\theta}]^T$, the matrix equation of the model could be linearized to yield a linear state-space model in the form of $\dot{\underset{\sim}{\mathbf{x}}} = \underline{\mathbf{A}}\underset{\sim}{\mathbf{x}} + \underline{\mathbf{B}} M(t)$, where explicitly is

$$\dot{\underset{\sim}{\mathbf{x}}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ \dfrac{m_A g l_A}{2\Delta} J_\theta & 0 & 0 & 0 \\ \dfrac{m_A g l_A}{2\Delta} L & 0 & 0 & 0 \end{bmatrix} \underset{\sim}{\mathbf{x}} + \begin{bmatrix} 0 \\ 0 \\ \dfrac{L}{\Delta} \\ \dfrac{J_\varphi}{\Delta} \end{bmatrix} M(t). \tag{2}$$

3 Control theory

Following previously done works on the Furuta pendulum [1, 3], I aim to apply the Linear Quadratic Regression (LQR) from optimal control theory [4, 5] to balance the pendulum.

3.1 Consideration on the use of LQR instead of PID control

In the case of the Furuta pendulum, the use of traditional PID control which treats each degree of freedom independently is problematic since *(i)* the system is strongly coupled, as shown by (1); *(ii)* the upright equilibrium is unstable. LQR control solves this problem by treating the coupled states as a multi-variable state simultaneously and optimizing a quantitative cost function for the selected priorities needed for the system.

3.2 Derivation of LQR theory

3.2.1 Infinite horizon continuous-time case

Consider the infinite horizon quadratic cost functional $J : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$ defined by

$$J[\underset{\sim}{\mathbf{x}}(\cdot), \underset{\sim}{\mathbf{u}}(\cdot)] = \int_0^\infty \left(\underset{\sim}{\mathbf{x}}^T \underline{\mathbf{Q}} \underset{\sim}{\mathbf{x}} + \underset{\sim}{\mathbf{u}}^T \underline{\mathbf{R}} \underset{\sim}{\mathbf{u}} \right) dt$$

where $\underset{\sim}{\mathbf{x}}$ satisfies the linear dynamics $\dot{\underset{\sim}{\mathbf{x}}} = \underline{\mathbf{A}} \underset{\sim}{\mathbf{x}} + \underline{\mathbf{B}} \underset{\sim}{\mathbf{u}}, \underset{\sim}{\mathbf{x}}(0) = \underset{\sim}{\mathbf{x}}_0$. Here, $\underset{\sim}{\mathbf{x}}(t) \in \mathbb{R}^n$ is the state trajectory of the system, $\underset{\sim}{\mathbf{u}}(t) \in \mathbb{R}^m$ denotes the control inputs of the system, $\underline{\mathbf{Q}} \in \mathcal{M}_n(\mathbb{R})$ is the weight state cost matrix which penalizes the deviation from intended state and $\underline{\mathbf{R}} \in \mathcal{M}_m(\mathbb{R})$ is the weight control cost matrix which penalizes for the effort used in control. Both $\underline{\mathbf{Q}}$ and $\underline{\mathbf{R}}$ are symmetric but $\underline{\mathbf{Q}}$ is semi-positive definite and $\underline{\mathbf{R}}$ is positive definite. We seek to minimize $J[\underset{\sim}{\mathbf{x}}, \underset{\sim}{\mathbf{u}}]$ by finding an optimal control law for the system.

Using Pontryagin's minimum principle [6], the control Hamiltonian is

$$H(\underset{\sim}{\mathbf{x}}, \underset{\sim}{\mathbf{u}}, \underset{\sim}{\boldsymbol{\lambda}}) = \underset{\sim}{\mathbf{x}}^T \underline{\mathbf{Q}} \underset{\sim}{\mathbf{x}} + \underset{\sim}{\mathbf{u}}^T \underline{\mathbf{R}} \underset{\sim}{\mathbf{u}} + \underset{\sim}{\boldsymbol{\lambda}}^T (\underline{\mathbf{A}} \underset{\sim}{\mathbf{x}} + \underline{\mathbf{B}} \underset{\sim}{\mathbf{u}})$$

where this changes the constrained optimization problem into an unconstrained one by introducing Lagrange multipliers $\underset{\sim}{\boldsymbol{\lambda}}(t) \in \mathbb{R}^n$, here given the name *costate vector*. It reflects the sensitivity of the optimal cost to the changes in the state. Optimality conditions from the minimum principle are as follows:

1. state equation
$$\dot{\underset{\sim}{\mathbf{x}}} = \frac{\partial H}{\partial \underset{\sim}{\boldsymbol{\lambda}}} = \underline{\mathbf{A}} \underset{\sim}{\mathbf{x}} + \underline{\mathbf{B}} \underset{\sim}{\mathbf{u}}$$

2. costate equation
$$\dot{\underset{\sim}{\boldsymbol{\lambda}}} = -\frac{\partial H}{\partial \underset{\sim}{\mathbf{x}}} = -2\underline{\mathbf{Q}} \underset{\sim}{\mathbf{x}} - \underline{\mathbf{A}}^T \underset{\sim}{\boldsymbol{\lambda}}$$

3. stationarity condition
$$\frac{\partial H}{\partial \underset{\sim}{\mathbf{u}}} = 2\underline{\mathbf{R}} \underset{\sim}{\mathbf{u}} + \underset{\sim}{\boldsymbol{\lambda}}^T \underline{\mathbf{B}} = \underset{\sim}{\mathbf{0}}$$

$$\implies \underset{\sim}{\mathbf{u}} = -\frac{1}{2}\underset{\sim}{\mathbf{R}}^{-1}\underset{\sim}{\boldsymbol{\lambda}}^T\underline{\mathbf{B}}$$

From here, we obtain the two-point boundary value problem (TPBVP) since $\underset{\sim}{\mathbf{x}}(t)$ evolves forward from $\underset{\sim}{\mathbf{x}}(t=0)$ while $\underset{\sim}{\boldsymbol{\lambda}}(t)$ evolves backward. To solve this TPBVP, the linear relationship

$$\underset{\sim}{\boldsymbol{\lambda}} = 2\underline{\mathbf{P}}\underset{\sim}{\mathbf{x}}$$

was assumed. Differentiating, we get

$$\dot{\underset{\sim}{\boldsymbol{\lambda}}} = 2\dot{\underline{\mathbf{P}}}\underset{\sim}{\mathbf{x}} + 2\underline{\mathbf{P}}\dot{\underset{\sim}{\mathbf{x}}}.$$

Using the state equation and the stationary condition and equating it to the costate equation, we arrive at

$$-2\underline{\mathbf{Q}}\underset{\sim}{\mathbf{x}} - \underline{\mathbf{A}}^T(2\underline{\mathbf{P}}\underset{\sim}{\mathbf{x}}) = 2\dot{\underline{\mathbf{P}}}\underset{\sim}{\mathbf{x}} + 2\underline{\mathbf{P}}\left(\underline{\mathbf{A}}\underset{\sim}{\mathbf{x}} - \underline{\mathbf{B}}\underline{\mathbf{R}}^{-1}\underline{\mathbf{B}}^T\underline{\mathbf{P}}\underset{\sim}{\mathbf{x}}\right).$$

For $\underset{\sim}{\mathbf{x}}_0 \neq \underset{\sim}{\mathbf{0}}$, simplifying and rearranging yields the differential Riccati equation:

$$-\dot{\underline{\mathbf{P}}} = \underline{\mathbf{A}}^T\underline{\mathbf{P}} + \underline{\mathbf{P}}\underline{\mathbf{A}} - \underline{\mathbf{P}}\underline{\mathbf{B}}\underline{\mathbf{R}}^{-1}\underline{\mathbf{B}}^T\underline{\mathbf{P}} + \underline{\mathbf{Q}}$$

Recalling that we are considering the infinite horizon case $t \to \infty$ since the objective is for the system to reach a steady state, so $\dot{\underline{\mathbf{P}}} = \underline{\mathbf{0}}$, and the equation simplifies to the algebraic Riccati equation:

$$\underline{\mathbf{A}}^T\underline{\mathbf{P}} + \underline{\mathbf{P}}\underline{\mathbf{A}} - \underline{\mathbf{P}}\underline{\mathbf{B}}\underline{\mathbf{R}}^{-1}\underline{\mathbf{B}}^T\underline{\mathbf{P}} + \underline{\mathbf{Q}} = \underline{\mathbf{0}} \tag{3}$$

Solving for $\underline{\mathbf{P}}$, which is unique and positive-definite for a stabilisable, detectable plant and substituting into the stationary condition [5, 7], the optimal control law is

$$\underset{\sim}{\mathbf{u}} = -\underbrace{\underline{\mathbf{R}}^{-1}\underline{\mathbf{B}}^T\underline{\mathbf{P}}}_{\underline{\mathbf{K}}}\underset{\sim}{\mathbf{x}} \tag{4}$$

where $\underline{\mathbf{K}}$ here is known as the *optimal feedback gain*.

3.2.2 Infinite horizon discrete-time case

While the continuous-time formulation provides the theoretical foundation, practical implementation on digital hardware requires a discrete-time equivalent. We now extend the derivation by converting the continuous integrals and derivatives into discrete sums and differences.

Consider the infinite horizon quadratic cost functional $J : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$ defined by

$$J[\underset{\sim}{\mathbf{x}}_k, \underset{\sim}{\mathbf{u}}_k] = \sum_{k=0}^{\infty}\left(\underset{\sim}{\mathbf{x}}_k^T\underline{\mathbf{Q}}\underset{\sim}{\mathbf{x}}_k + \underset{\sim}{\mathbf{u}}_k^T\underline{\mathbf{R}}\underset{\sim}{\mathbf{u}}_k\right)$$

where $\underset{\sim}{\mathbf{x}}_k$ satisfies the discrete linear dynamics $\underset{\sim}{\mathbf{x}}_{k+1} = \underline{\mathbf{A}}\underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{B}}\underset{\sim}{\mathbf{u}}_k, \underset{\sim}{\mathbf{x}}_0 = \underset{\sim}{\mathbf{x}}(0)$. Here, $\underset{\sim}{\mathbf{x}}_k \in \mathbb{R}^n$ is the state vector of the system at step k, $\underset{\sim}{\mathbf{u}}_k \in \mathbb{R}^m$ denotes the control inputs also at step k, both $\underline{\mathbf{Q}} \in \mathcal{M}_n(\mathbb{R})$ and $\underline{\mathbf{R}} \in \mathcal{M}_m(\mathbb{R})$ are the same quantities defined

previously in the continuous case. We seek to minimize $J[\underset{\sim}{\mathbf{x}}, \underset{\sim}{\mathbf{u}}]$ by finding an optimal control law for the system.

Following similarly to the continuous case, using the discrete-time Pontryagin's Minimum Principle, the control Hamiltonian is

$$H_k(\underset{\sim}{\mathbf{x}}_k, \underset{\sim}{\mathbf{u}}_k, \underset{\sim}{\boldsymbol{\lambda}}_{k+1}) = \underset{\sim}{\mathbf{x}}_k^T \underline{\mathbf{Q}} \underset{\sim}{\mathbf{x}}_k + \underset{\sim}{\mathbf{u}}_k^T \underline{\mathbf{R}} \underset{\sim}{\mathbf{u}}_k + \underset{\sim}{\boldsymbol{\lambda}}_{k+1}^T (\underline{\mathbf{A}} \underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{B}} \underset{\sim}{\mathbf{u}}_k)$$

where this changes the constrained optimization problem into an unconstrained one by introducing Lagrange multipliers (costate vectors) $\underset{\sim}{\boldsymbol{\lambda}}_k \in \mathbb{R}^n$ at step k, . Similar optimality conditions from the minimum principle are obtained as follows:

1. state equation
$$\underset{\sim}{\mathbf{x}}_{k+1} = \frac{\partial H_k}{\partial \underset{\sim}{\boldsymbol{\lambda}}_{k+1}} = \underline{\mathbf{A}} \underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{B}} \underset{\sim}{\mathbf{u}}_k$$

2. costate equation
$$\underset{\sim}{\boldsymbol{\lambda}}_k = \frac{\partial H_k}{\partial \underset{\sim}{\mathbf{x}}_k} = 2\underline{\mathbf{Q}} \underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{A}}^T \underset{\sim}{\boldsymbol{\lambda}}_{k+1}$$

3. stationarity condition
$$\frac{\partial H_k}{\partial \underset{\sim}{\mathbf{u}}_k} = 2\underline{\mathbf{R}} \underset{\sim}{\mathbf{u}}_k + \underline{\mathbf{B}}^T \underset{\sim}{\boldsymbol{\lambda}}_{k+1} = \underset{\sim}{\mathbf{0}}$$
$$\implies \underset{\sim}{\mathbf{u}}_k = -\frac{1}{2} \underline{\mathbf{R}}^{-1} \underline{\mathbf{B}}^T \underset{\sim}{\boldsymbol{\lambda}}_{k+1}$$

From here, we obtain the discrete two-point boundary value problem (TPBVP) since $\underset{\sim}{\mathbf{x}}_k$ evolves forward from $\underset{\sim}{\mathbf{x}}_0$ while $\underset{\sim}{\boldsymbol{\lambda}}_k$ evolves backward. Similarly, to solve this TPBVP, the linear relationship

$$\underset{\sim}{\boldsymbol{\lambda}}_k = 2\underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_k$$

was assumed. Note that this also implies $\underset{\sim}{\boldsymbol{\lambda}}_{k+1} = 2\underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_{k+1}$. Substituting this into the stationarity condition yields

$$\underset{\sim}{\mathbf{u}}_k = -\underline{\mathbf{R}}^{-1} \underline{\mathbf{B}}^T \underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_{k+1} = -\underline{\mathbf{R}}^{-1} \underline{\mathbf{B}}^T \underline{\mathbf{P}} (\underline{\mathbf{A}} \underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{B}} \underset{\sim}{\mathbf{u}}_k).$$

Solving for $\underset{\sim}{\mathbf{u}}_k$, we obtain the discrete optimal control law:

$$\underset{\sim}{\mathbf{u}}_k = -(\underline{\mathbf{R}} + \underline{\mathbf{B}}^T \underline{\mathbf{P}} \underline{\mathbf{B}})^{-1} \underline{\mathbf{B}}^T \underline{\mathbf{P}} \underline{\mathbf{A}} \underset{\sim}{\mathbf{x}}_k.$$

Substituting $\underset{\sim}{\boldsymbol{\lambda}}_k = 2\underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_k$ and $\underset{\sim}{\boldsymbol{\lambda}}_{k+1} = 2\underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_{k+1}$ into the costate equation gives

$$2\underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_k = 2\underline{\mathbf{Q}} \underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{A}}^T (2\underline{\mathbf{P}} \underset{\sim}{\mathbf{x}}_{k+1}).$$

Substituting the state equation and the expression for $\underset{\sim}{\mathbf{u}}_k$, simplifying, and rearranging yields the discrete algebraic Riccati equation [8, 9]:

$$\underline{\mathbf{P}} = \underline{\mathbf{Q}} + \underline{\mathbf{A}}^T \underline{\mathbf{P}} \underline{\mathbf{A}} - \underline{\mathbf{A}}^T \underline{\mathbf{P}} \underline{\mathbf{B}} (\underline{\mathbf{R}} + \underline{\mathbf{B}}^T \underline{\mathbf{P}} \underline{\mathbf{B}})^{-1} \underline{\mathbf{B}}^T \underline{\mathbf{P}} \underline{\mathbf{A}}. \tag{5}$$

Recalling that we are considering the infinite horizon case $k \to \infty$ since the objective is for the system to reach a steady state, $\underline{\mathbf{P}}$ converges to a constant matrix. The optimal feedback control law is then given by

$$\underset{\sim}{\mathbf{u}}_k = -\underline{\mathbf{K}}\underset{\sim}{\mathbf{x}}_k, \quad \text{where} \quad \underline{\mathbf{K}} = (\underline{\mathbf{R}} + \underline{\mathbf{B}}^T\underline{\mathbf{P}}\underline{\mathbf{B}})^{-1}\underline{\mathbf{B}}^T\underline{\mathbf{P}}\underline{\mathbf{A}}. \tag{6}$$

Note that the additional term $\underline{\mathbf{B}}^T\underline{\mathbf{P}}\underline{\mathbf{B}}$ as compared to the continuous-time case in (4) is to account for the fact that $\underset{\sim}{\mathbf{u}}_k$ affects the state $\underset{\sim}{\mathbf{x}}_{k+1}$ before the cost is evaluated at the next step.

3.3 Solving for the optimal control law

Having done the derivation for both cases above, it could be seen that both cases end in a matrix equation for $\underline{\mathbf{P}}$ for which no closed-form solution exists. Because of this, a fixed-point iteration [10] could be applied to iteratively solve it; direct spectral methods exist [7] and are used here only as an independent check. This subsection will describe that iteration and the properties of it that the implementation relies on.

3.3.1 The sampled plant

The design is carried out on the sampled plant rather than on the continuous pair $(\underline{\mathbf{A}}, \underline{\mathbf{B}})$ of (2), because the moment is held constant by the motor driver between controller updates [11]. At control period Δt the zero-order-hold equivalent of the linearised model is

$$\underline{\mathbf{A}}_d = e^{\underline{\mathbf{A}}\Delta t}, \qquad \underline{\mathbf{B}}_d = \int_0^{\Delta t} e^{\underline{\mathbf{A}}\sigma}\underline{\mathbf{B}}\, d\sigma = \sum_{k=0}^{\infty} \frac{\underline{\mathbf{A}}^k\underline{\mathbf{B}}\,\Delta t^{k+1}}{(k+1)!},$$

so the state propagates as $\underset{\sim}{\mathbf{x}}_{k+1} = \underline{\mathbf{A}}_d\underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{B}}_d M_k$ with $M_k = M(t_k)$, and the cost being minimised is the discrete one of §3.2.2, with $\underline{\mathbf{Q}}$ and $\underline{\mathbf{R}}$ read as weights per sample.

The distinction is not cosmetic at the control rate used. Solving the continuous problem of §3.2.1 with the same weights returns a controller whose bandwidth is not attainable at the control rate, whereas the sampled problem returns a markedly gentler gain; the two agree only in the limit $\Delta t \to 0$, so the control law implemented is the discrete one of §3.2.2 throughout. The two designs are compared quantitatively in §4.5.1.

3.3.2 The Riccati map and Riccati iteration

Recall that we only have a single input, $m = 1$ so $\underset{\sim}{\mathbf{u}} = [M(t)]$, and with the state in §2.2, $\underset{\sim}{\mathbf{x}} = [\varphi \quad \theta \quad \dot{\varphi} \quad \dot{\theta}]^T$, I define the control weighting is a scalar function $s(\underline{\mathbf{P}})$ and the gain row $\underline{\mathbf{F}}(\underline{\mathbf{P}})$ is

$$s(\underline{\mathbf{P}}) = \underline{\mathbf{R}} + \underline{\mathbf{B}}_d^T\underline{\mathbf{P}}\underline{\mathbf{B}}_d \in \mathbb{R}, \qquad \underline{\mathbf{F}}(\underline{\mathbf{P}}) = s(\underline{\mathbf{P}})^{-1}\underline{\mathbf{B}}_d^T\underline{\mathbf{P}}\underline{\mathbf{A}}_d \in \mathbb{R}^{1\times 4}.$$

In terms of $\underline{\mathbf{F}}$ the discrete algebraic Riccati equation of §3.2.2 reads

$$\mathcal{R}(\underline{\mathbf{P}}) = \underline{\mathbf{Q}} + \underline{\mathbf{A}}_d^T\underline{\mathbf{P}}\,(\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d\underline{\mathbf{F}}(\underline{\mathbf{P}}))\,,$$

which is that equation itself, because

$$\underline{\mathbf{A}}_d^T\underline{\mathbf{P}}\underline{\mathbf{B}}_d\underline{\mathbf{F}}(\underline{\mathbf{P}}) = \underline{\mathbf{A}}_d^T\underline{\mathbf{P}}\underline{\mathbf{B}}_d\left(\underline{\mathbf{R}} + \underline{\mathbf{B}}_d^T\underline{\mathbf{P}}\underline{\mathbf{B}}_d\right)^{-1}\underline{\mathbf{B}}_d^T\underline{\mathbf{P}}\underline{\mathbf{A}}_d.$$

Hence $\underline{\mathbf{P}} = \mathcal{R}(\underline{\mathbf{P}})$ holds if and only if $\underline{\mathbf{P}}$ solves the discrete algebraic Riccati equation, and the optimal gain corresponding to that $\underline{\mathbf{P}}$ is $\underline{\mathbf{K}} = \underline{\mathbf{F}}(\underline{\mathbf{P}})$. Then, the problem is can be shown as the fixed point of the coupled recursion

$$\underline{\mathbf{P}}_{k+1} = \mathcal{R}(\underline{\mathbf{P}}_k), \qquad \underline{\mathbf{K}}_k = \underline{\mathbf{F}}(\underline{\mathbf{P}}_k), \qquad \underline{\mathbf{P}}_0 = \underline{\mathbf{Q}},$$

and the control law applied is $\underset{\sim}{\mathbf{u}}_k = -\underline{\mathbf{K}}\underset{\sim}{\mathbf{x}}_k$ with $\underline{\mathbf{K}} = \lim_{k\to\infty} \underline{\mathbf{K}}_k$. The initial condition $\underline{\mathbf{P}}_0 = \underline{\mathbf{Q}}$ is the recursion that a finite-horizon problem with one step to go would produce, so the infinite-horizon solution is reached as the horizon is extended. Each iteration is a single evaluation of $\mathcal{R}$: at $n = 4$, this leads to $O(n^3)$ complexity, and no matrix inverse is needed since $s(\underline{\mathbf{P}})$ is scalar for $m = 1$. This iteration is also known as the *Riccati iteration*.

3.3.3 Convergence and stability of the iteration

The following could be concluded from the properties of the Riccati map $\mathcal{R}$ and the iteration, due to the algebraic properties of the matrices involved:

1. (*Monotonicity*) The map $\mathcal{R}$ is monotone with respect to the positive semi-definite order: $\underline{\mathbf{P}} \preceq \underline{\mathbf{P}}' \Rightarrow \mathcal{R}(\underline{\mathbf{P}}) \preceq \mathcal{R}(\underline{\mathbf{P}}')$. Starting from $\underline{\mathbf{P}}_0 = \underline{\mathbf{Q}} \succeq \underline{\mathbf{0}}$ the iterates are therefore non-decreasing.

2. (*Boundedness*) Since $(\underline{\mathbf{A}}_d, \underline{\mathbf{B}}_d)$ is controllable, as the controllability matrix of (2) has rank 4, and $\underline{\mathbf{Q}} \succ \underline{\mathbf{0}}$ for the weights used, the sequence is bounded above by the stabilising solution $\underline{\mathbf{P}}_+$.

A monotone bounded sequence in the positive semi-definite order converges [8], and its limit is the unique stabilising solution of the discrete algebraic Riccati equation [9].

Stability of the iterative solution could be checked by the nature of the matrix corresponding to the state of the system, obtained from the dynamics of the system and the control law applied.:

$$\begin{aligned}
\underset{\sim}{\mathbf{x}}_{k+1} &= \underline{\mathbf{A}}_d\underset{\sim}{\mathbf{x}}_k + \underline{\mathbf{B}}_d\underset{\sim}{\mathbf{u}}_k \\
&= \underline{\mathbf{A}}_d\underset{\sim}{\mathbf{x}}_k - \underline{\mathbf{B}}_d\underline{\mathbf{K}}_k\underset{\sim}{\mathbf{x}}_k \\
&= (\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d\underline{\mathbf{K}}_k)\underset{\sim}{\mathbf{x}}_k
\end{aligned}$$

The system is stable if and only if the real part of all the eigenvalues of the closed loop matrix $\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d\underline{\mathbf{K}}_k$ are negative, or equivalently, if the spectral radius $\rho(\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d\underline{\mathbf{K}}_k) < 1$, such that the limit $\underline{\mathbf{K}} = \lim_{k\to\infty} \underline{\mathbf{K}}_k$ must satisfy this condition, that is the solution of the discrete algebraic Riccati equation must be asympototically stable. The iteration should therefore run to convergence and never truncated for speed, since intermediate iterates may not yet stabilise the plant. A phrase to say that this limit is stable is "closed-loop poles inside the unit disc." The measurements confirming each of these properties for the default design are reported in §4.7.1.

3.3.4 Termination

Since $\underline{\mathbf{P}}$ scales with $\underline{\mathbf{Q}}/\underline{\mathbf{R}}$, an absolute stopping rule is not meaningful: the magnitude of $\underline{\mathbf{P}}$ at the solution varies by orders of magnitude across the range of weights exposed by the implementation. The implemented test is therefore relative,

$$\|\underline{\mathbf{P}}_{k+1} - \underline{\mathbf{P}}_k\|_{\max} \;\le\; \tau \max\left(1, \|\underline{\mathbf{P}}_k\|_{\max}\right), \qquad \tau = 10^{-9},$$

with a cap of $k_{\max} = 8000$ iterations, and the returned gain is $\underline{\mathbf{F}}$ evaluated at the final iterate. The $\max(1, \cdot)$ factor keeps the test meaningful when the weights themselves are small.

Because $\underline{\mathbf{F}}$ is a smooth function of $\underline{\mathbf{P}}$, an accuracy of order τ in $\underline{\mathbf{P}}$ carries over to $\underline{\mathbf{K}}$: the converged gain agrees with a direct solution of the discrete algebraic Riccati equation [7] to the order of the tolerance, and the residual of the Riccati equation at the returned $\underline{\mathbf{P}}$ is $\mathcal{O}(\tau)$ relative to $\|\underline{\mathbf{P}}\|_{\max}$, as the stopping rule prescribes. The iteration count, however, is not fixed: it tracks τ and grows as the control period shortens, since the recursion converges more slowly as the closed-loop spectral radius approaches unity. The cap is therefore a guard against a control period short enough that the recursion is still unconverged, not a tuning parameter — the iteration should be left to converge rather than truncated. The measured counts, the solver agreement and the behaviour of the cap against Δt are reported in §4.7.2.

4 Software implementation

The controller designed above is realised in the JavaScript engine `js/physics.js`, together with the application layer in `js/simulator.js`, which drives the companion interactive simulator. The engine has no dependencies, no matrix library and no build step: every matrix involved here is at most 4×4, so matrices are stored as arrays of rows and multiplied by explicit triple loops. This section documents what the code computes, in the order in which it computes it, and quotes the numbers it produces for the default hardware of §2.2 — the slider defaults of the simulator, namely $l_A = 0.25$ m, $l_B = 0.20$ m, $m_A = 0.08$ kg, $m_B = 0.12$ kg and $g = 9.81$ m/s^2, with the bars treated as uniform slender rods.

4.1 Software layout

function	responsibility
`deriveParams`	hardware $\rightarrow$ $J_\theta, J_\varphi, B, L$, with ${}^{A}J_1 = {}^{A}J_2 = m_A l_A^2/3$, ${}^{B}J_3 = m_B l_B^2/3$ and the axial term ${}^{A}J_3$ set to the negligible regularising value 10^{-6}
`accel`, `accelLagrange`	the nonlinear EoMs (1) in the matrix form $\underline{\mathbf{M}}\ddot{\underset{\sim}{\mathbf{q}}} + \underline{\mathbf{C}}\dot{\underset{\sim}{\mathbf{q}}} = \underset{\sim}{\tau}$ and in the raw Lagrange form, the latter kept for cross-validation
`stepRK4`	one classical Runge–Kutta step of the nonlinear model
`energy`, `pendEnergy`	$T + V$, and the swing-up energy with datum at the upright
`linStateSpace`	$(\underline{\mathbf{A}}, \underline{\mathbf{B}})$ of (2)
`expm`, `discretizeZOH`	$(\underline{\mathbf{A}}_d, \underline{\mathbf{B}}_d)$ at the control period
`lqrDiscrete`, `lqrGains`	the Riccati iteration and the complete design pipeline
`controller`, `swingTorque`, `bangTorque`	the finite-state controller and the two swing-up laws
`sensor`, `lowpass`	encoder noise and velocity filtering

The dynamic state is carried as a plain object whose keys are `th`, `ph`, `thd` and `phd`, and every consumer reads its components by name. No permutation between an input ordering and a state ordering can therefore be introduced by accident: the gain row solved in §3.3.2 is applied component by component in the mathematical ordering $\underset{\sim}{\mathbf{x}} = [\varphi, \theta, \dot{\varphi}, \dot{\theta}]^T$ fixed in §2.2. The file `js/simulator.js` owns the clock, the finite-state wiring, the sliders, the charts and the telemetry, and `test/physics.test.js` exercises the engine headlessly under Node.

4.2 Linearisation

`linStateSpace` assembles $(\underline{\mathbf{A}}, \underline{\mathbf{B}})$ of (2) directly, without ever forming the nonlinear mass matrix: it computes $\Delta = J_\theta J_\varphi - L^2$, then $k = (m_A g l_A/2)/\Delta$, places the two unit entries $A_{13} = A_{24} = 1$, the gravity coefficients $(A_{31}, A_{41}) = k\,(J_\theta, L)$ and $\underline{\mathbf{B}} = (0, 0, L/\Delta, J_\varphi/\Delta)^T$. The inertia-variation amplitude B does not appear, since it multiplies $\sin^2\varphi$ and vanishes at the upright equilibrium. For the default hardware the derived constants are

quantity	expression	value
J_θ	$m_A l_B^2 + m_B l_B^2/4 + {}^B J_3 + {}^A J_3$	6.0010×10^{-3}
J_φ	$m_A l_A^2/4 + {}^A J_1$	2.9167×10^{-3}
B	$m_A l_A^2/4 + {}^A J_2 - {}^A J_3$	2.9157×10^{-3}
L	$m_A l_A l_B/2$	2.0000×10^{-3}
Δ	$J_\theta J_\varphi - L^2$	1.3503×10^{-5}

and the resulting linear model is

$$\underline{\mathbf{A}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 43.598 & 0 & 0 & 0 \\ 14.530 & 0 & 0 & 0 \end{bmatrix}, \qquad \underline{\mathbf{B}} = \begin{bmatrix} 0 \\ 0 \\ 148.12 \\ 216.00 \end{bmatrix}.$$

4.3 Zero-order-hold discretisation

`discretizeZOH` evaluates the two series of §3.3.1 with `expm` and a term recursion. The matrix exponential is computed by scaling and squaring with a Taylor series [12, 13]: the argument $\underline{\mathbf{A}}\Delta t$ is halved until $\|\cdot\|_{\max} \le 0.05$ (three halvings for the default $\underline{\mathbf{A}}$ at $\Delta t = 5$ ms, since $\|\underline{\mathbf{A}}\Delta t\|_{\max} = 43.598 \times 5 \times 10^{-3} = 0.218$), the series $\sum_k \underline{\mathbf{A}}^k/k!$ is then accumulated up to 39 terms with an early exit once a term falls below 10^{-16}, and the result is squared back the same number of times. At that scale the series reaches machine precision in about seven terms, so the loop terminates long before its limit. The input series for $\underline{\mathbf{B}}_d$ — an integral of the matrix exponential [14] — is accumulated in the same style from $k = 0$ to $k = 59$, reusing successively higher powers of $\underline{\mathbf{A}}$, with the factorial evaluated in closed form; it also terminates after a few terms. The result at the control period $\Delta t = 5$ ms is

$$\underline{\mathbf{A}}_d = \begin{bmatrix} 1.000545 & 0 & 5.00091 \times 10^{-3} & 0 \\ 1.8164 \times 10^{-4} & 1 & 3.0273 \times 10^{-7} & 5 \times 10^{-3} \\ 0.218029 & 0 & 1.000545 & 0 \\ 0.0726642 & 0 & 1.8164 \times 10^{-4} & 1 \end{bmatrix},$$

$$\underline{\mathbf{B}}_d = \begin{bmatrix} 1.85162 \times 10^{-3} \\ 2.70009 \times 10^{-3} \\ 0.740715 \\ 1.08006 \end{bmatrix},$$

which satisfies the expected small-period check $\underline{\mathbf{B}}_d \to \underline{\mathbf{B}}\Delta t = (0,\ 0,\ 0.740580,\ 1.080014)^T$ to four significant figures, and $\underline{\mathbf{A}}_d \to \underline{\mathbf{I}} + \underline{\mathbf{A}}\Delta t$ likewise.

4.4 The Riccati iteration in code

The solver is a single function, `lqrDiscrete`, transcribed below with the middle columns of the loop collapsed (the helper calls `mmul`, `madd`, `msub`, `mtrans` and `mmaxabs` are the obvious row-major matrix operations; an unused identity is elided). The comments give the corresponding quantity of §3.3.2.

```
function lqrDiscrete(Ad, Bd, Q, R, maxIter, tol) {
  maxIter = maxIter || 8000;
  tol = tol || 1e-9;
  let P = Q.map(r => r.slice());          // P_0 = Q (copied)
```

```
  const BT = mtrans(Bd);
  for (let it = 0; it < maxIter; it++) {
    const PB = mmul(P, Bd);                // P B_d
    const S = R + mmul(BT, PB)[0][0];      // s(P), scalar (m = 1)
    const AtP = mmul(mtrans(Ad), P);       // A_d' P
    const BtPA = mmul(BT, mmul(P, Ad));    // B_d' P A_d
    const Fv = BtPA[0].map(v => v / S);    // F(P) = K_k
    const BF = mmul(Bd, [Fv]);             // B_d F
    const Pnew = madd(mmul(AtP, msub(Ad, BF)), Q);   // R(P_k)
    const scale = Math.max(1, mmaxabs(P));
    if (mmaxabs(msub(Pnew, P)) < tol * scale) {
      return { K: Fv, P: Pnew, it };       // converged
    }
    P = Pnew;
  }
  // cap reached: recompute K from the last iterate and return it
  const BtPA = mmul(BT, mmul(P, Ad))[0];
  const BtPB = mmul(BT, mmul(P, Bd))[0][0];
  return { K: BtPA.map(v => v / (R + BtPB)), P, it: maxIter };
}
```

Read against §3.3.2, the loop body is the map $\mathcal{R}$ itself: `PB` is $\underline{\mathbf{P}}\underline{\mathbf{B}}_d$, `S` is the scalar $s(\underline{\mathbf{P}})$, `BtPA` is $\underline{\mathbf{B}}_d^T\underline{\mathbf{P}}\underline{\mathbf{A}}_d$, `Fv` is $\underline{\mathbf{F}}(\underline{\mathbf{P}})$, `BF` is $\underline{\mathbf{B}}_d\underline{\mathbf{F}}(\underline{\mathbf{P}})$ and `Pnew` is $\underline{\mathbf{Q}} + \underline{\mathbf{A}}_d^T\underline{\mathbf{P}}(\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d\underline{\mathbf{F}}(\underline{\mathbf{P}}))$. Three consequences of $m = 1$ are visible in the listing: $s(\underline{\mathbf{P}})$ is a plain number so no matrix inverse is ever formed, the gain row is a flat array of four numbers, and $\underline{\mathbf{K}}$ is built from the same intermediate products as $\underline{\mathbf{P}}_{k+1}$, so the iteration returns the gain at no extra cost.

Two choices deserve comment. First, the stopping test is relative with a floor of unity, exactly the rule of §3.3.4, so one value of τ behaves identically for the default weights and for the light-weight corner of the slider range. Second, if the cap is reached the function recomputes $\underline{\mathbf{F}}$ from the last iterate and returns it instead of failing; that fallback is not converged, and as noted in §3.3.4 it is reached only for control periods far shorter than the 5 ms used here.

4.5 Design pipeline and the two designs

`lqrGains` is the three-step pipeline:

`linStateSpace` → `discretizeZOH` → `lqrDiscrete`,

it is the only entry point the application uses `recomputeGains` in `js/simulator.js` calls it twice — once per design — on startup and again on every change of a weight slider or of a hardware slider, redisplaying both gains. For the two designs used:

design	$\underline{\mathbf{Q}}$	R	iterations	closed-loop poles
balance	$\mathrm{diag}(300, 25, 5, 1.5)$	0.012	629	$0.0027;\ 0.9633;\ 0.9850 \pm 0.0076i$
capture	$\mathrm{diag}(500, 1, 8, 0.1)$	0.008	1533	$0.0018;\ 0.9613;\ 0.9944 \pm 0.0048i$

with gains

$$\underline{\mathbf{K}}_{bal} = [\,21.5636 \quad -2.2748 \quad 3.5956 \quad -1.5111\,],$$
$$\underline{\mathbf{K}}_{cap} = [\,14.4030 \quad -0.4586 \quad 2.1603 \quad -0.5337\,],$$

both in the ordering of §2.2, and both with every closed-loop pole inside the unit disc. The capture design is deliberately indifferent to the arm angle — its weight on θ is 500 times smaller than its weight on φ — because it is used only while the pendulum is being caught and the arm is allowed to run; the balance design then brings θ back to zero.

4.5.1 The control rate and the design

The choice of the sampled problem over the continuous one, argued on physical grounds in §3.3.1, is worth quantifying, because at the control rate used the two are far apart. Continuing with the balance weights $\underline{\mathbf{Q}} = \mathrm{diag}(300, 25, 5, 1.5)$ and $R = 0.012$, solving the continuous problem of §3.2.1 gives

$$\underline{\mathbf{K}} \approx [\,419.98 \quad -45.64 \quad 70.11 \quad -30.10\,],$$

a controller whose slowest poles sit at $\approx -3.0 \pm 1.5i$ rad/s, whereas the sampled problem at $\Delta t = 5$ ms returns

$$\underline{\mathbf{K}} \approx [\,21.56 \quad -2.27 \quad 3.60 \quad -1.51\,],$$

which is markedly gentler, since a bandwidth of that order is not attainable at 200 Hz. The two agree only in the limit $\Delta t \to 0$: already at $\Delta t = 0.1$ ms the same weights return a gain fifteen times larger in its first entry, close to the continuous value. This is the quantitative statement of the argument of §3.3.1, and it is why the control law implemented is the discrete one of §3.2.2 throughout.

4.6 Control loop implementation and energy-pumping swing up

The nonlinear model is integrated with RK4 at 1 kHz ($\Delta t = 1$ ms) while the controller runs at 200 Hz ($\Delta t_c = 5$ ms), that is one control update every fifth physics step, so the moment is held constant over each control interval — the same zero-order hold that §3.3.1 assumed when the design was discretised.

At each update the measured state is formed from the true state by `sensor`: Gaussian encoder noise, applied as 0.5 level on the two angles and 2.5 level on the two rates (the interface default is 0.01 rad and the filter is off by default), followed by a first-order low-pass on both rates with $\alpha = 0.2$. The angles are wrapped to $(-\pi, \pi]$, and the torque is

```
const K = (ctrl.Kp && aPhi > ctrl.phiSafe) ? ctrl.Kp : ctrl.K;
M = -(K[0]*phiW + K[1]*thW + K[2]*s.phd + K[3]*s.thd);
const lim = ctrl.Mmax;
if (lim > 0) M = Math.max(-lim, Math.min(lim, M));
```

so the gain row switches from the balance design to the capture design while $|\varphi| > 0.12$ rad, and the result is saturated at $\pm M_{max} = 0.5$ N·m (the slider allows 0.1–1.2 N·m). The saturation is the one part of the arrangement outside the LQR problem, which assumes an unbounded input; it is a hard nonlinearity, and it is a bound the balance controller is deliberately tested against.

Outside the capture zone the torque comes from the energy-pumping swing-up law rather than from the gain [15, 16]:

$$E = \frac{1}{2} J_\varphi \dot{\varphi}^2 + m_A g \frac{l_A}{2} (\cos\varphi - 1), \qquad M = M_{max}\,\mathrm{sgn}\,(-E\,\dot{\varphi}\cos\varphi) - k_{arm}\dot{\theta},$$

that is, saturated torque in whichever direction increases the energy towards $E_{ref} = 0$, plus an arm-damping term with $k_{arm} = 0.03$ that stops θ from winding up. The proportional alternative $M = -k_E E \operatorname{sgn}(\dot{\varphi} \cos \varphi)$ is also implemented and exposed through the k_E slider, but the saturated form is the default because it reaches the capture zone fastest while remaining inside the torque limit. With the basis convention of §2.1.1 one has $\dot{E} \propto \dot{\varphi} \cos \varphi \cdot M$ to leading order, so the leading minus sign is what makes $E \to E_{ref}$: when the pendulum lacks energy the arm accelerates in the direction of the pendulum's swing.

Because LQR is only valid near the upright equilibrium, the two laws are combined by a finite-state machine with hysteresis [3]. The swing-up phase ends when $|\varphi| \leq 0.30$ rad with $|\dot{\varphi}| \leq 2.5$ rad/s and $|\dot{\theta}| \leq 3.0$ rad/s; balance is then held until the pendulum is clearly lost at $|\varphi| > 0.55$ rad, which prevents the two phases from chattering while an aggressive catch is in progress. The capture bound and the torque limit are exposed as sliders, alongside k_{arm}, k_E and the sensor noise level, and the robustness runs of §4.7 leave them at their defaults, since a pendulum that enters the capture zone too fast or too far off-vertical is the one failure mode the linear design cannot survive.

4.7 Verification of the default gains

This subsection collects the measurements that check the default designs against the theory of §3. It is organised by what is being checked: the properties of the Riccati iteration used in §3.3.3, the stopping rule of §3.3.4 together with an independent solve of the Riccati equation, and finally the plant and controller behaviour of the assembled simulator.

4.7.1 Synthesiser properties

The three claims of §3.3.3 were tested by running the iteration on the default design.

- *Monotonicity.* The symmetric part of $\underline{\mathbf{P}}_{k+1} - \underline{\mathbf{P}}_k$ remained positive semi-definite to within 4×10^{-12} over 700 iterations of the default design.

- *The limit stabilises the plant.* The closed-loop matrix $\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d \underline{\mathbf{K}}$ has every eigenvalue inside the unit disc; for the default weights they are 0.0027, 0.9633 and $0.9850 \pm 0.0076i$, matching the balance row of the table in §4.5. The pendulum mode is nearly deadbeat at the control rate while the arm mode stays slow, which is the expected split for a pendulum-first weighting.

- *Only the limit may be used.* Intermediate iterates do not stabilise the plant: for the default design $\rho(\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d \underline{\mathbf{F}}_k) \approx 1.018$ over the first $\sim$100 iterations and crosses below unity only near convergence. The iteration is therefore run to convergence and never truncated for speed.

- *Rate.* Writing $\rho = \rho(\underline{\mathbf{A}}_d - \underline{\mathbf{B}}_d \underline{\mathbf{K}})$, the distance to the limit contracts by a factor ρ^2 per iteration; measured over iterations 201–600 of the default design the contraction is 0.97048, against $\rho^2 = 0.98507^2 = 0.97035$. Equivalently, one decade of accuracy costs $[-\log_{10} \rho^2]^{-1} = 76.5$ iterations.

4.7.2 Termination and agreement with a direct solve

The stopping rule of §3.3.4 was exercised across its whole range. $\|\underline{\mathbf{P}}\|_{\max}$ is 3.870×10^{4} for the default weights but 2.26×10^{2} for $R=5$ with the lightest weights, so an absolute rule would need retuning per design while the relative rule does not.

The iteration count tracks the tolerance the way the rate of §4.7.1 predicts — 179 iterations at $\tau=10^{-3}$, 421 at 10^{-6}, 629 at 10^{-9} and 883 at 10^{-12}, roughly 76 iterations bought by each further decade. For the default weights the recursion terminates after 629 iterations at

$$\underline{\mathbf{K}} = [\,21.5636 \quad -2.2748 \quad 3.5956 \quad -1.5111\,],$$

and the control applied is $M_k=-\underline{\mathbf{K}}\underline{\tilde{\mathbf{x}}}_k$, saturated at the motor limit; the second design, weighted far more heavily on the pendulum than on the arm, is used at the capture hand-off and needs 1533 iterations, again as listed in §4.5. Across the whole range of weights exposed by the implementation ($q_\varphi\in[10,3000]$, $q_\theta\in[0,200]$, $q_{\dot\varphi}\in[0,60]$, $q_{\dot\theta}\in[0,30]$, $R\in[0.002,5]$) the count stays between 163 and 1533, so the cap is never approached at the control period used.

It is not unlimited comfort: at a tenfold faster control rate ($\Delta t=1$ ms) the same design needs 2927 iterations, and at $\Delta t=0.1$ ms the cap is reached while the recursion is still unconverged — the returned gain then differs from the converged one by ≈ 16 in max-norm — which the fixed control period of 5 ms avoids. This is the sense in which $k_{\max}$ is a guard rather than a tuning parameter, as stated in §3.3.4.

The solved gain was finally checked against an independent solution of the discrete algebraic Riccati equation that shares none of the iteration's numerics. The Riccati iteration approaches $\underline{\mathbf{P}}$ as a fixed point of the Riccati map; the reference instead forms the symplectic pencil of the problem and extracts the stabilising solution from its stable deflating subspace, which is the ordered Schur method of Laub [7] and the approach used by the dense solver `scipy.linalg.solve_discrete_are`. The two are compared on identical inputs, $\underline{\mathbf{A}}_d$ and $\underline{\mathbf{B}}_d$ being taken from the engine itself, so only the algorithm differs.

For the default design the gains agree to 5.5×10^{-7} in max-norm, a relative difference of 2.6×10^{-8}, and the capture design to 7.7×10^{-7}. The residual of the Riccati equation at the $\underline{\mathbf{P}}$ returned by the iteration is 3.7×10^{-5}, that is 9.6×10^{-10} relative to $\|\underline{\mathbf{P}}\|_{\max}$ as the stopping rule prescribes, while the reference solution achieves 5.1×10^{-10}, hence the residual is within the tolerance defined previously. The closed-loop poles match those listed in §4.7.1 and every one lies inside the unit disc. This check is reproducible: `test/dare-crosscheck.py` runs it, and `test/physics.test.js` invokes it as test `[9b]`, skipping gracefully where SciPy is unavailable.

4.7.3 Plant and controller tests

The engine carries its own two formulations of the dynamics where in the file `test/physics.test.js`, the matrix form and the raw Lagrange form of (1) agree to 10^{-9} over 200 random states; with $M=0$ the RK4 integration conserves the mechanical energy $T+V$ to better than 10^{-4} relative over 10 s.

At the controller level the same test file runs the finite-state machine headlessly: balance holds for 10 s from an initial offset of 0.12 rad without the pendulum ever exceeding 0.15 rad, a swing-up from hanging captures in under 8 s and then holds for

40 s, six different initial states all swing up and balance without the arm winding up, a sharp kick of 1.5 rad/s applied at $t = 6$ s is rejected, noisy sensors at 0.01 rad still capture and hold, and five different hardware variants — lighter and heavier pendulums, longer and shorter arms — all swing up and balance with the same gains. The browser end-to-end suite finally drives the same engine through the simulator page, checking that the gains are genuinely recomputed when a weight changes, that the swing-up from hanging is captured, that a kick given while balancing is rejected, and that the system is still balancing after 30 s of simulated time at four times real speed.

Acknowledgements

I would like to thank my Intro. to Dynamics and Vibrations lecturer, Prof. Robert G. Parker for his encouragement when I first brought up the idea of analyzing stability of dynamical systems. Deepseek AI was used in the generation of the initial draft of the report and the implementation of the code in Javascript. I have verified the correctness of the content and made necessary modifications to ensure accuracy and clarity.

References

[1] K. Furuta, M. Yamakita, and S. Kobayashi, "Swing-up control of inverted pendulum using pseudo-state feedback," *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 206, no. 4, pp. 263–269, 1992.

[2] R. G. Parker, *Essential Dynamics*. 2026.

[3] I. Fantoni and R. Lozano, *Non-linear Control for Underactuated Mechanical Systems*. Communications and Control Engineering, Springer London, 2002.

[4] R. E. Kalman, "Contributions to the theory of optimal control," *Boletín de la Sociedad Matemática Mexicana, II. Ser.*, vol. 5, pp. 102–119, 1960.

[5] B. D. O. Anderson and J. B. Moore, *Optimal control. Linear quadratic methods*. Prentice Hall Information and System Sciences Series, Englewood Cliffs, NJ: Prentice Hall, 1990. Dover reprint, 2007, ISBN 978-0-486-45766-6.

[6] L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko, *The Mathematical Theory of Optimal Processes*. New York: Interscience Publishers, 1962.

[7] A. Laub, "A Schur method for solving algebraic Riccati equations," *IEEE Transactions on Automatic Control*, vol. 24, no. 6, pp. 913–921, 1979.

[8] P. E. Caines and D. Q. Mayne, "On the discrete time matrix Riccati equation of optimal control," *International Journal of Control*, vol. 12, no. 5, pp. 785–794, 1970. Correction note: ibid. 14(1):205–207, 1971, doi 10.1080/00207177108932032.

[9] B. Molinari, "The stabilizing solution of the discrete algebraic Riccati equation," *IEEE Transactions on Automatic Control*, vol. 20, no. 3, pp. 396–399, 1975.

[10] D. Kleinman, "On an iterative technique for Riccati equation computations," *IEEE Transactions on Automatic Control*, vol. 13, no. 1, pp. 114–115, 1968.

[11] K. J. Åström and B. Wittenmark, *Computer-Controlled Systems: Theory and Design*. Dover Publications, 3rd ed., 2011. Also published by Prentice Hall, ISBN 0-13-314899-8.

[12] C. Moler and C. Van Loan, "Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later," *SIAM Review*, vol. 45, no. 1, pp. 3–49, 2003.

[13] N. J. Higham, "The scaling and squaring method for the matrix exponential revisited," *SIAM Journal on Matrix Analysis and Applications*, vol. 26, no. 4, pp. 1179–1193, 2005.

[14] C. Van Loan, "Computing integrals involving the matrix exponential," *IEEE Transactions on Automatic Control*, vol. 23, no. 3, pp. 395–404, 1978.

[15] K. J. Åström and K. Furuta, "Swinging up a pendulum by energy control," *Automatica*, vol. 36, no. 2, pp. 287–295, 2000.

[16] M. W. Spong, "Energy based control of a class of underactuated mechanical systems," *IFAC Proceedings Volumes*, vol. 29, no. 1, pp. 2828–2832, 1996.